

The Mathematics of the Music Microscope

Course Notes for the Audio-Visualizer Project

Harry Geng

Living document — grows with the codebase.
Chapters marked *(outline)* are planned but not yet written.

July 10, 2026

Contents

Preface	iii
I Signals	1
1 From Sound to Vectors	2
1.1 Sampling	2
1.2 The Discrete Fourier Transform	3
1.3 The Fast Fourier Transform	4
1.4 Frames, Windows, and the Spectrogram	4
1.5 Pitch by Autocorrelation	5
2 Filters, Onsets, and Beats (<i>outline</i>)	7
II Learning	8
3 Attention and Transformers (<i>outline</i>)	9
4 Self-Supervised Audio Representations: MERT (<i>outline</i>)	10
5 Source Separation (<i>outline</i>)	11
6 Contrastive Embeddings: CLAP and the Vibe Axes (<i>outline</i>)	12
III Geometry of Timbre	13
7 Maps of Timbre Space: PCA and UMAP	14
7.1 The Data: Moments on a Sphere	14
7.2 Principal Component Analysis	15
7.3 UMAP I: The Graph	16
7.4 UMAP II: The Layout	17
7.5 What the Map Does and Does Not Mean	18
7.6 Color as a Second Opinion	18
8 Similarity Search and the Moment Index (<i>outline</i>)	20

IV	Rendering	21
9	The Mathematics of Rendering (<i>outline</i>)	22
A	Equal Temperament and Chroma	24
A.1	Pitch is Logarithmic	24

Preface

These notes document, with proofs where they are illuminating, the mathematics running inside the Audio-Visualizer project: a system that ingests songs, separates them into stems with neural networks, extracts musical structure, embeds every few seconds of audio into learned vector spaces, and renders the result as interactive visualizations.

The material spans four traditions that rarely share a book: classical signal processing (Part I), modern deep learning (Part II), the geometry of high-dimensional data (Part III), and computer graphics (Part IV). The unifying thread is that *every object defined here actually runs*: each chapter ends where the repository begins, and the “In the code” boxes point at the specific file, function, and parameter values where a definition earns its keep. When a theorem’s hypothesis is violated in practice (they often are), the notes say so and describe what breaks.

Exercises range from short proofs to experiments you can run against your own music library. None are busywork; several reproduce bugs that were actually encountered while building the system.

How to read this

Chapter 1 and Chapter 7 are complete and set the standard for the rest. The remaining chapters exist as detailed outlines and will be filled in as the corresponding subsystems stabilize. Prerequisites are first-year linear algebra and calculus; everything else is developed as needed.

Notation

$x[n]$	a discrete-time signal (square brackets: integer index)
f_s	sampling rate in Hz
$X[k]$	the DFT of $x[n]$; k indexes frequency bins
$\ x\ $	Euclidean norm; $\ x\ _F$ the Frobenius norm
$\langle x, y \rangle$	inner product $x^\top y$
S^{d-1}	the unit sphere $\{x \in \mathbb{R}^d : \ x\ = 1\}$
$A^\top$	matrix transpose
$\mathbf{1}[\cdot]$	indicator function
$\log$	natural logarithm unless a base is written

Part I

Signals

Chapter 1

From Sound to Vectors

Everything in this project begins by turning pressure waves into arrays of numbers, and arrays of numbers into frequencies. This chapter builds the discrete Fourier transform from scratch, proves the identities the rest of the book leans on, and ends at the two workhorse measurements the visualizer makes sixty times per second: the spectrogram and the autocorrelation pitch estimate.

1.1 Sampling

A microphone measures a continuous function $x_c : \mathbb{R} \rightarrow \mathbb{R}$ (air pressure against time). Computers store countably many numbers, so we sample.

Definition 1.1 (Sampling). Given a continuous signal x_c and a *sampling rate* $f_s > 0$ (in Hz), the sampled signal is the sequence

$$x[n] = x_c\left(\frac{n}{f_s}\right), \quad n \in \mathbb{Z}.$$

Sampling destroys information in general: infinitely many continuous signals agree on any given sample sequence. The classical theorem says that *bandlimited* signals are the exception.

Theorem 1.2 (Nyquist–Shannon). *Let x_c be bandlimited to B Hz, meaning its continuous Fourier transform vanishes outside $[-B, B]$. If $f_s > 2B$, then x_c is uniquely determined by its samples $x[n]$, and can be reconstructed as*

$$x_c(t) = \sum_{n \in \mathbb{Z}} x[n] \operatorname{sinc}(f_s t - n), \quad \operatorname{sinc}(u) = \frac{\sin(\pi u)}{\pi u}.$$

Proof idea. Bandlimitedness means the Fourier transform $\hat{x}_c$ is supported on $[-B, B]$. Sampling in time corresponds to *periodizing* in frequency: the transform of the sample train is $\sum_k \hat{x}_c(f - kf_s)$, a sum of copies of $\hat{x}_c$ spaced f_s apart. If $f_s > 2B$ the copies do not overlap, so multiplying by the indicator of $[-f_s/2, f_s/2]$ recovers $\hat{x}_c$ exactly; that multiplication in frequency is convolution with the sinc kernel in time. $\square$

When $f_s \leq 2B$ the copies overlap and distinct frequencies become indistinguishable after sampling — *aliasing*: a tone at frequency f is captured identically to one at $f_s - f$.

In the code

The project stores two versions of every stem (`ingest.py`): a listening copy at $f_s = 44,100$ Hz stereo (Nyquist frequency 22.05 kHz, above human hearing) and an analysis copy at $f_s = 22,050$ Hz mono (`config.py`: `SR = 22050`), which is cheaper to process and still covers content up to 11 kHz — plenty for beat, pitch, and chroma analysis. Every resample (e.g. to MERT’s 24 kHz in `embeddings.py`, or CLAP’s 48 kHz in `backfill_clap.py`) must low-pass first for exactly the reason in Theorem 1.2; `librosa.resample` does this internally.

1.2 The Discrete Fourier Transform

Fix a frame length N and view a chunk of signal as a vector $x = (x[0], \dots, x[N-1]) \in \mathbb{C}^N$.

Definition 1.3 (DFT). Let $\omega_N = e^{-2\pi i/N}$. The *discrete Fourier transform* of $x \in \mathbb{C}^N$ is $X \in \mathbb{C}^N$ with

$$X[k] = \sum_{n=0}^{N-1} x[n] \omega_N^{kn} = \sum_{n=0}^{N-1} x[n] e^{-2\pi i kn/N}, \quad k = 0, \dots, N-1.$$

The DFT is the change of basis into the family of sampled complex sinusoids $e_k[n] = e^{2\pi i kn/N}$. That family is orthogonal:

Lemma 1.4 (Orthogonality). For $k, \ell \in \{0, \dots, N-1\}$,

$$\sum_{n=0}^{N-1} e^{2\pi i (k-\ell)n/N} = \begin{cases} N & k = \ell, \\ 0 & k \neq \ell. \end{cases}$$

Proof. If $k = \ell$ every term is 1. Otherwise let $q = e^{2\pi i (k-\ell)/N} \neq 1$; the sum is geometric: $\sum_{n=0}^{N-1} q^n = \frac{q^N - 1}{q - 1}$, and $q^N = e^{2\pi i (k-\ell)} = 1$, so the numerator vanishes. $\square$

Corollary 1.5 (Inversion and Parseval). $x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{2\pi i kn/N}$, and

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2.$$

Proof. Substitute Definition 1.3 into the claimed inverse and swap the two finite sums; Lemma 1.4 collapses the inner sum to $N \mathbf{1}[n = m]$. Parseval follows by expanding $|X[k]|^2 = X[k] \overline{X[k]}$ and applying the lemma once more. $\square$

Parseval is the statement that *energy is preserved*: what the ear hears as loudness can be measured either per-sample or per-frequency-bin. The visualizer uses both sides of this identity (RMS level on the time side, band energy on the frequency side).

Remark 1.6 (What a bin means). Bin k of an N -point DFT at sampling rate f_s represents frequency

$$f_k = \frac{k f_s}{N} \quad \text{Hz}, \quad k \leq N/2,$$

so the frequency resolution is $\Delta f = f_s/N$: longer frames see finer pitch differences. For a real signal, $X[N-k] = \overline{X[k]}$, so only the first $N/2 + 1$ bins carry information.

In the code

The chroma computation in `scene.js` (`computeChroma`) inverts Remark 1.6: it walks the analyser’s bins and maps each to its frequency via `fr = bin * sr / nfft` before folding onto pitch classes (Appendix A). With `fftSize = 1024` at $f_s = 44.1$ kHz, $\Delta f \approx 43$ Hz — coarse at the bottom of the piano (the gap between A1 and B♭1 is only 3.3 Hz) and luxurious at the top. This asymmetry between linear bins and logarithmic musical pitch recurs throughout the project.

1.3 The Fast Fourier Transform

Computing Definition 1.3 literally costs N multiplications for each of N bins: $\Theta(N^2)$. The FFT gets the same numbers in $\Theta(N \log N)$.

Theorem 1.7 (Cooley–Tukey). *Let N be even, and split x into even- and odd-indexed halves with DFTs E and O (each of length $N/2$). Then for $k = 0, \dots, N/2 - 1$:*

$$X[k] = E[k] + \omega_N^k O[k], \quad X[k + N/2] = E[k] - \omega_N^k O[k].$$

Consequently, for $N = 2^m$, the DFT can be computed in $\Theta(N \log N)$ operations.

Proof. Split the defining sum by parity of n , writing $n = 2r$ and $n = 2r + 1$:

$$X[k] = \sum_{r=0}^{N/2-1} x[2r] \omega_N^{2rk} + \omega_N^k \sum_{r=0}^{N/2-1} x[2r+1] \omega_N^{2rk} = E[k] + \omega_N^k O[k],$$

using $\omega_N^2 = \omega_{N/2}$. The second identity follows from $\omega_N^{k+N/2} = -\omega_N^k$. The cost recurrence $T(N) = 2T(N/2) + \Theta(N)$ solves to $\Theta(N \log N)$ by the master theorem. $\square$

This is why every frame size in the project is a power of two (`fftSize` of 1024, 2048): the browser’s `AnalyserNode` and NumPy’s FFT both rely on this factorization.

1.4 Frames, Windows, and the Spectrogram

Music is not stationary — the whole point is that its frequency content changes. The fix is to transform short *frames* and accept a tradeoff.

Definition 1.8 (STFT and spectrogram). Fix a frame length N , a hop $H \leq N$, and a window $w : \{0, \dots, N-1\} \rightarrow [0, 1]$. The *short-time Fourier transform* of x is

$$X[m, k] = \sum_{n=0}^{N-1} x[mH + n] w[n] e^{-2\pi i k n / N},$$

and the *spectrogram* is the matrix of squared magnitudes $S[m, k] = |X[m, k]|^2$.

Without a window ($w \equiv 1$), chopping the signal introduces artificial discontinuities at frame edges, and Definition 1.3 smears each true frequency across all bins (*spectral leakage*). A smooth taper such as the Hann window

$$w[n] = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{N-1} \right)$$

concentrates that leakage into nearby bins at the price of slightly widening each spectral peak.

Remark 1.9 (Time–frequency tradeoff). $\Delta f = f_s/N$ improves with longer frames while time localization $\Delta t = N/f_s$ worsens *in exact inverse proportion*: $\Delta t \cdot \Delta f = 1$ regardless of N . This is the discrete shadow of the uncertainty principle. There is no window that beats it; one can only choose where to spend it. Musical consequence: a system tuned to see bass notes precisely (long frames) necessarily blurs drum onsets, and vice versa.

In the code

The microscope’s spectrogram view (`microscope.py`, `spectrogram.png`) renders $\log(1 + S)$ rather than S — the log compresses the enormous dynamic range of music (~ 60 dB between a hi-hat tail and a kick) into the few dozen brightness levels an eye can distinguish, then maps it through the magma colormap. The scene mode’s live analysis instead uses the browser’s `AnalyserNode` (`app.js`, `bands()`), which computes Definition 1.8 with a built-in Blackman window at `fftSize = 1024`: $\Delta t \approx 23$ ms, $\Delta f \approx 43$ Hz — the “spend it on time” end of Remark 1.9, correct for a visualizer chasing drum hits.

1.5 Pitch by Autocorrelation

The spectrogram spreads a note’s energy across its harmonics; for *monophonic* sources (one note at a time — bass, a single voice) a time-domain method finds the fundamental more directly.

Definition 1.10 (Autocorrelation). The (unnormalized, windowed) autocorrelation of x at lag $\tau \geq 0$ over a window of length W is

$$r[\tau] = \sum_{n=0}^{W-1} x[n] x[n + \tau].$$

Proposition 1.11 (Periodicity peaks). Suppose x is P -periodic on the relevant range, i.e. $x[n + P] = x[n]$. Then $r[\tau] \leq r[0]$ for all τ , with equality at every multiple of P : $r[0] = r[P] = r[2P] = \dots$

Proof. Equality at multiples of P is immediate from periodicity. For the bound, Cauchy–Schwarz gives

$$r[\tau] = \langle x_{0:W}, x_{\tau:\tau+W} \rangle \leq \|x_{0:W}\| \|x_{\tau:\tau+W}\| = r[0],$$

where the final equality uses periodicity again (both windows contain the same multiset of values when W is a whole number of periods). $\square$

So the algorithm is: compute $r[\tau]$ over a plausible range of lags, find the peak, and report $f_0 = f_s/\tau^*$. Two practical failure modes matter. First, r peaks equally at $2P, 3P, \dots$, so noise can make the estimator jump down an octave (choosing $2P$) — the classic *octave error*. Second, on silence or noise the global maximum is meaningless, so an energy gate must precede the search.

In the code

`Transport.pitchHz` in `app.js` is Definition 1.10 verbatim: window $W = 512$ samples, lag range $f_s/320 \leq \tau \leq f_s/50$ (i.e. candidate fundamentals between 50 and 320 Hz — the playable range of a bass guitar), and an RMS gate of 0.012 before searching. The stutter you once saw in the bass fret marker was octave error plus frame-to-frame noise; the mitigations in `scene.js` (sampling every third frame, level-gating, exponential smoothing 0.7/0.3) treat the symptoms. The planned cure — an offline pitch tracker with Viterbi smoothing across the

whole song — replaces the per-frame $\arg \max$ with a globally optimal path and will live in the ingest pipeline.

Exercises

Exercise 1.1. A pure 9 kHz tone is sampled at $f_s = 16$ kHz with no anti-aliasing filter. What frequency is heard on playback? Generalize: give the perceived frequency as a function of true frequency f and f_s .

Exercise 1.2. Prove the conjugate-symmetry claim of Remark 1.6: if $x[n] \in \mathbb{R}$ for all n , then $X[N - k] = \overline{X[k]}$. How many real numbers are needed to store the DFT of a real signal, and why does this not contradict the DFT being invertible?

Exercise 1.3. Complete the proof of Corollary 1.5 in full detail, justifying the interchange of summations.

Exercise 1.4. Show that the Hann window satisfies $\sum_{m \in \mathbb{Z}} w[n - mH] = \text{const}$ when $H = N/2$ (perfect overlap-add at 50% hop). Why does this property matter for reconstructing audio from a modified STFT — for instance, in mask-based source separation (Chapter 5)?

Exercise 1.5 (computational). Record or load a bass stem from your library. Implement Definition 1.10 and plot $r[\tau]$ for one frame during a held note. Identify the fundamental's peak and at least one octave-error peak. Then break the estimator on purpose: find a frame where the $\arg \max$ lands at $2P$.

Exercise 1.6. Using Remark 1.9: the microscope wants to distinguish $E1$ (41.2 Hz) from $F1$ (43.7 Hz) in the bass register. What minimum frame length (in samples at 22,050 Hz, and in milliseconds) does a plain DFT need? Roughly how many kick drum hits at 170 BPM occur within one such frame? Conclude something about why the project analyzes bass pitch and drum onsets with different tools.

Chapter 2

Filters, Onsets, and Beats (*outline*)

How the project isolates frequency bands, detects the instant a drum is struck, and infers a tempo grid from a noisy sequence of onsets.

In the code

Status: outline. To be written against `feature_extractor.py` (onset envelopes, beat tracking) and the legacy Butterworth drum-band split that DrumSep replaced.

Planned contents:

- **LTI systems and convolution.** Impulse response, frequency response as the DFT of the kernel; why filtering is multiplication in frequency (convolution theorem, with proof).
- **Butterworth filters.** Maximally-flat magnitude response $|H(\omega)|^2 = 1/(1 + (\omega/\omega_c)^{2n})$; biquad implementation; phase distortion and why the legacy kick/snare/hat split smeared transients.
- **Onset detection.** Spectral flux $\sum_k \max(0, |X[m, k]| - |X[m-1, k]|)$; logarithmic compression; adaptive thresholding (moving median) — and the scene-mode variant: the per-piece peak-normalized onset gate in `scene.js` that lets a quiet hi-hat register next to a loud kick.
- **Beat tracking as dynamic programming.** Tempo estimation via the autocorrelation of the onset envelope; the Ellis DP recurrence $D(t) = O(t) + \max_\tau (D(t - \tau) - \lambda \log^2(\tau/\tau_0))$; backtracking; why breakdowns and rubato break it.
- **Sections and phrases.** Self-similarity matrices over chroma and timbre features; novelty kernels; the checkerboard detector.

Part II

Learning

Chapter 3

Attention and Transformers (*outline*)

The workhorse architecture behind every neural network in this project: MERT, CLAP’s text tower, Demucs’ bottleneck, and BS-RoFormer are all transformers. This chapter builds the mechanism once so later chapters can use it freely.

In the code

Status: outline. The definitions here are prerequisites for Chapters 4–6.

Planned contents:

- **Scaled dot-product attention.** $\text{Attn}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) V$; attention as a data-dependent convex combination; the $\sqrt{d_k}$ scaling justified via the variance of dot products of random vectors (with proof).
- **Multi-head attention** as subspace projections; parameter counting; why heads specialize.
- **Positional information.** Sinusoidal encodings; relative position; *rotary embeddings* (RoPE) — rotation matrices acting on pairs of coordinates, and the theorem that RoPE makes attention scores a function of relative offset only (this is the “Ro” in BS-RoFormer).
- **The full block.** Residual streams, LayerNorm (with the gradient argument for pre-norm), feed-forward expansion; $O(T^2)$ cost in sequence length and what that means for 75 fps audio frames.
- **Softmax and temperature.** Properties used throughout: invariance to additive shifts, the Jacobian, low/high temperature limits.

Chapter 4

Self-Supervised Audio Representations: MERT (*outline*)

How a network learns what music sounds like with no labels at all: mask part of the signal, force the model to predict it, and the representations needed to succeed turn out to be the ones we wanted.

In the code

Status: outline. To be written against `embeddings.py` (`m-a-p/MERT-v1-95M`, frame embeddings at ~ 75 fps, 13 hidden layers averaged, mean-pooled per moment).

Planned contents:

- **The masked-prediction template.** BERT-style objectives for continuous signals; why predicting raw waveforms fails and targets must be discretized or abstracted.
- **Discrete targets via vector quantization.** Codebooks, the RVQ (residual VQ) construction used by neural codecs; commitment losses; MERT’s acoustic teacher.
- **MERT’s musical teacher.** The constant-Q transform (CQT): logarithmically-spaced filters aligned with equal temperament (Appendix A); why adding a CQT reconstruction loss injects pitch awareness that codec tokens miss.
- **The convolutional front end.** Strided convolutions as learned filterbanks; receptive field and frame-rate arithmetic.
- **Which layer knows what.** Probing studies; why the pipeline averages all 13 hidden states rather than taking the last; mean-pooling as a bag-of-frames assumption and what temporal information it forfeits.
- **From frames to moments.** The pooling map $e = \frac{1}{T} \sum_t h_t$ and its effect on the geometry of Chapter 7.

Chapter 5

Source Separation (*outline*)

The signature capability of the whole project: given a stereo mixture, recover drums, bass, vocals, and the rest. Mathematically: invert a many-to-one map, using learned priors about what instruments sound like.

In the code

Status: outline. To be written against `ingest.py`: Demucs (`htdemucs`, `htdemucs_6s`), BS-RoFormer vocals, the MDX23C DrumSep cascade, and the measured $\sim 17\times$ vocal-bleed reduction from the RoFormer-first cascade.

Planned contents:

- **The problem is ill-posed.** Mixture $m = \sum_s x_s$; the solution set is an affine subspace; separation = choosing by prior.
- **Mask-based separation.** Time-frequency masks $\hat{X}_s = M_s \odot X_m$; the ideal ratio mask; phase reuse and its artifacts; why overlap-add consistency (Exercise 1.4) matters.
- **Waveform models and hybrids.** Demucs’ U-Net encoder/decoder with a transformer bottleneck operating in both time and spectrogram domains; skip connections as multiresolution identity paths.
- **Band-split RoFormer.** Partitioning the spectrogram into perceptually-sized bands, per-band tokens, RoPE attention across time and band axes (Chapter 3); why this architecture currently wins vocals benchmarks.
- **Evaluation.** SDR/SIR/SAR decomposition, in decibels; what a +1 dB SDR gain sounds like; why “piano” is hard (spectral overlap with *everything* — the boundary-flicker documented in the project memory).
- **Cascades.** Removing a well-modeled source first (vocals) shrinks the residual problem for the rest; conditions under which cascading helps vs. compounds errors, with the project’s drum-kit split as the worked example.

Chapter 6

Contrastive Embeddings: CLAP and the Vibe Axes (*outline*)

One model, two encoders, one shared space: CLAP aligns audio with language, which lets us name directions in sound-space using English sentences. The vibe visualizations are driven by dot products against those directions.

In the code

Status: outline. To be written against `backfill_clap.py` (audio tower), `compute_vibe.py` (text axes: bright/warm/dense/tense/euphoric/vast), and `microscope.py`'s `/api/textsearch`.

Planned contents:

- **The InfoNCE objective.** Batch of (a_i, t_i) pairs;

$$\mathcal{L} = -\frac{1}{B} \sum_i \log \frac{\exp(\langle a_i, t_i \rangle / \tau)}{\sum_j \exp(\langle a_i, t_j \rangle / \tau)};$$

symmetric audio→text and text→audio terms; the learned temperature τ ; InfoNCE as a lower bound on mutual information (statement, proof sketch).

- **Geometry of the shared space.** Why both towers L2-normalize; alignment vs. uniformity decomposition of the loss; the modality gap (audio and text occupy offset cones) and why *differences* of text embeddings partially cancel it.
- **Zero-shot classification = nearest text.** Text search over moments as a single matrix-vector product (`emb @ text_vec`).
- **Semantic axes.** The vibe construction: multi-prompt pole averaging (variance reduction), $\text{score} = \langle e, v_+ \rangle - \langle e, v_- \rangle$; percentile normalization as a monotone calibration; why axes are *named* but not *orthogonal* — correlated axes (bright/euphoric) and what that implies for palette mappings.
- **Failure modes.** Prompt sensitivity; the broken-checkpoint incident (`laion/larger_clap_music` collapsed embeddings) as a case study in validating learned components empirically.

Part III

Geometry of Timbre

Chapter 7

Maps of Timbre Space: PCA and UMAP

The galaxy view draws every moment of every song as a star, and similar-sounding moments land near each other. This chapter contains the complete mathematics of that map: from 768-dimensional learned embeddings, through an optimal linear compression (PCA), to a nonlinear graph embedding (UMAP) whose axes provably mean nothing — and whose neighborhoods mean everything.

7.1 The Data: Moments on a Sphere

Chapter 4 describes how MERT turns each ~ 4 -second moment of audio into a vector $e \in \mathbb{R}^{768}$ by mean-pooling transformer states. Here we take those vectors as given. The pipeline first normalizes:

$$x = \frac{e}{\|e\|} \in S^{767},$$

placing every moment on the unit sphere. This makes similarity purely *angular*:

Definition 7.1 (Cosine similarity and distance). For nonzero $u, v \in \mathbb{R}^d$,

$$\text{sim}(u, v) = \frac{\langle u, v \rangle}{\|u\| \|v\|}, \quad d_{\cos}(u, v) = 1 - \text{sim}(u, v) \in [0, 2].$$

For unit vectors, $\text{sim}(u, v) = \langle u, v \rangle$ and $\|u - v\|^2 = 2d_{\cos}(u, v)$: cosine distance and squared Euclidean distance are the same thing on the sphere, up to a factor of 2.

Remark 7.2. $d_{\cos}$ is not a metric (it fails the triangle inequality in general), but the angle $\arccos(\text{sim})$ is. In practice every algorithm below only ever *ranks* distances, for which the distinction is harmless.

In the code

`compute_galaxy.py` stacks the `emb_mix` arrays of every song into $X \in \mathbb{R}^{n \times 768}$ and normalizes rows (`X /= np.linalg.norm(X, axis=1, keepdims=True)`). With the current local library, $n = 664$; the PC batch library will push n into the tens of thousands, which is where every guarantee in this chapter gets sharper.

7.2 Principal Component Analysis

The first reduction $768 \rightarrow 64$ is linear, classical, and optimal in two senses that turn out to be the same sense.

Let $X \in \mathbb{R}^{n \times d}$ have centered columns (subtract the mean row $\bar{x}$ first), and define the sample covariance

$$C = \frac{1}{n-1} X^\top X \in \mathbb{R}^{d \times d}.$$

C is symmetric positive semidefinite, so by the spectral theorem it has an orthonormal eigenbasis $v_1, \dots, v_d$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$.

Theorem 7.3 (Variance maximization). *The direction of maximal projected variance,*

$$\arg \max_{\|u\|=1} \text{Var}(Xu) = \arg \max_{\|u\|=1} u^\top C u,$$

is v_1 , and the maximum equals λ_1 . More generally, the k -dimensional subspace maximizing total projected variance is $\text{span}(v_1, \dots, v_k)$, capturing variance $\sum_{i=1}^k \lambda_i$.

Proof. Expand u in the eigenbasis: $u = \sum_i \alpha_i v_i$ with $\sum_i \alpha_i^2 = 1$. Then

$$u^\top C u = \sum_i \lambda_i \alpha_i^2 \leq \lambda_1 \sum_i \alpha_i^2 = \lambda_1,$$

with equality iff $\alpha_i = 0$ whenever $\lambda_i < \lambda_1$. For the subspace statement, apply the same argument inductively on the orthogonal complement of the span chosen so far (the Courant–Fischer characterization). $\square$

Theorem 7.4 (Best low-rank approximation, Eckart–Young). *Write the (thin) singular value decomposition $X = U \Sigma V^\top$ with singular values $\sigma_1 \geq \dots \geq \sigma_d$. Among all matrices M of rank at most k ,*

$$\|X - M\|_F^2 \geq \sum_{i>k} \sigma_i^2,$$

with equality for $M = U \Sigma_k V^\top$ (Σ_k keeps the top k singular values). Equivalently: projecting the rows of X onto $\text{span}(v_1, \dots, v_k)$ is the rank- k linear map minimizing total squared reconstruction error.

Remark 7.5. Theorems 7.3 and 7.4 are the same fact seen from two sides, connected by $C = \frac{1}{n-1} V \Sigma^2 V^\top$: the covariance eigenvectors are the right singular vectors, and $\lambda_i = \sigma_i^2 / (n-1)$. “Keep the most variance” and “lose the least reconstruction” pick the same subspace.

In the code

`PCA(n_components=64).fit_transform(X)` in `compute_galaxy.py`. Two engineering motives, neither about visualization per se: (i) UMAP’s neighbor search runs $\sim 12\times$ faster in 64 dimensions than in 768; (ii) the trailing dimensions of mean-pooled transformer embeddings are mostly noise, and Theorem 7.4 says truncation is the *least* destructive way to drop them. Note that with $n = 664 < 768$ the data matrix cannot have rank above 663 anyway; PCA here is denoising, not compression.

7.3 UMAP I: The Graph

UMAP¹ operates in two acts. Act one converts the point cloud $\{x_1, \dots, x_n\} \subset \mathbb{R}^{64}$ into a weighted graph encoding local structure; act two lays that graph out in the plane. The high-dimensional coordinates never appear again after act one — everything the map knows is in the graph.

Definition 7.6 (*k*-NN graph). Fix $k \in \mathbb{N}$ (here $k = 15$) and a metric d (here $d_{\cos}$). For each i let $N(i)$ be the set of its k nearest neighbors. The directed graph has an edge $i \rightarrow j$ for each $j \in N(i)$.

Exact k -NN costs $\Theta(n^2d)$; the implementation uses NN-descent, a randomized refinement scheme, for near-linear approximate search.

The heart of UMAP is what it does with the raw distances next. Two constants are computed per point:

Definition 7.7 (Local calibration). For each point i , set $\rho_i = \min_{j \in N(i)} d(x_i, x_j)$ (distance to the nearest neighbor), and let $\sigma_i > 0$ solve

$$\sum_{j \in N(i)} \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right) = \log_2 k. \quad (7.1)$$

The directed membership strength of edge $i \rightarrow j$ is then

$$w_{i \rightarrow j} = \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right).$$

Lemma 7.8 (The calibration is well-posed). *Suppose not all neighbors of i are at distance exactly ρ_i (generic position). Then Equation (7.1) has a unique solution $\sigma_i \in (0, \infty)$.*

Proof. Let $g(\sigma)$ denote the left side, a finite sum of terms $\exp(-c_j/\sigma)$ with constants $c_j = \max(0, d_{ij} - \rho_i) \geq 0$, at least one $c_j = 0$ (the nearest neighbor) and, by assumption, at least one $c_j > 0$. Each term is continuous and nondecreasing in σ ; terms with $c_j > 0$ are strictly increasing. Hence g is continuous and strictly increasing, with

$$\lim_{\sigma \rightarrow 0^+} g(\sigma) = \#\{j : c_j = 0\} \geq 1, \quad \lim_{\sigma \rightarrow \infty} g(\sigma) = k.$$

Since $1 \leq \log_2 k < k$ for $k \geq 2$ (here $\log_2 15 \approx 3.91$), the intermediate value theorem gives existence whenever $\#\{j : c_j = 0\} \leq \log_2 k$, and strict monotonicity gives uniqueness. (The implementation solves (7.1) by binary search.) $\square$

Remark 7.9 (Why calibrate?). Equation (7.1) forces every point to distribute the same total membership mass ($\log_2 k$) over its neighborhood. In a dense region, σ_i comes out small (distances are shrunk aggressively); in a sparse region, large. This is the “Uniform” in UMAP: *assume* the data is uniformly distributed on some manifold, and absorb observed density variation into a point-dependent rescaling of distance. The ρ_i shift additionally guarantees every point is connected with weight 1 to its nearest neighbor, so no point is orphaned. The cost of this assumption: UMAP largely erases density information — a tight cluster and a loose cluster of the same membership look alike on the map. Cluster *size* on the galaxy is not evidence of anything.

¹McInnes, Healy, Melville, *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction* (2018). These notes present the algorithm as implemented, with the category-theoretic framing omitted.

Directed weights disagree ($w_{i \rightarrow j} \neq w_{j \rightarrow i}$), so they are merged:

Definition 7.10 (Fuzzy union). The symmetric weight of the pair $\{i, j\}$ is

$$\mu_{ij} = w_{i \rightarrow j} + w_{j \rightarrow i} - w_{i \rightarrow j} w_{j \rightarrow i}.$$

Remark 7.11. If the two directed weights are read as independent probabilities that “ i accepts j as a neighbor” and vice versa, μ_{ij} is exactly $\Pr(\text{at least one accepts})$ — the probabilistic OR (t -conorm). One-sided friendships survive; that keeps bridges between regions alive.

7.4 UMAP II: The Layout

Act two seeks low-dimensional points $y_1, \dots, y_n \in \mathbb{R}^2$ whose *own* fuzzy graph resembles μ . The low-dimensional membership is defined by a smooth rational kernel:

Definition 7.12 (Low-dimensional kernel). Fix $a, b > 0$. For $y_i, y_j \in \mathbb{R}^2$,

$$\nu_{ij} = \left(1 + a \|y_i - y_j\|^{2b}\right)^{-1}.$$

The constants a, b are fit by least squares so that ν approximates

$$\Psi(t) = \begin{cases} 1 & t \leq \text{min_dist}, \\ \exp(-(t - \text{min_dist})) & t > \text{min_dist}, \end{cases}$$

i.e. “full membership within `min_dist`, exponential decay beyond.”

`min_dist` is therefore a pure *rendering* parameter: it sets how tightly the layout may pack points that the graph considers identical, trading cluster crispness against within-cluster detail.

Definition 7.13 (Objective). The layout minimizes the fuzzy cross-entropy

$$\mathcal{C}(y) = \sum_{i < j} \underbrace{\mu_{ij} \log \frac{\mu_{ij}}{\nu_{ij}}}_{\text{attraction}} + \underbrace{(1 - \mu_{ij}) \log \frac{1 - \mu_{ij}}{1 - \nu_{ij}}}_{\text{repulsion}}.$$

The names are earned by the gradients: the first term is minimized by making ν_{ij} large (pull connected points together), the second by making ν_{ij} small (push unconnected points apart). The final map is the equilibrium of $n(n-1)/2$ simultaneous tugs-of-war.

Proposition 7.14 (Attractive gradient). *For the attractive term of a single pair, with $r = \|y_i - y_j\|$,*

$$\frac{\partial}{\partial y_i} \left[-\mu_{ij} \log \nu_{ij} \right] = \mu_{ij} \frac{2ab r^{2(b-1)}}{1 + ar^{2b}} (y_i - y_j),$$

a spring whose stiffness decays as points approach — unlike a Hookean spring, distant neighbors are pulled hard while satisfied ones are left alone.

Proof. $-\log \nu_{ij} = \log(1 + ar^{2b})$; apply the chain rule with $\partial r^2 / \partial y_i = 2(y_i - y_j)$ and simplify. $\square$

Minimization proceeds by stochastic gradient descent over *edges*: each epoch samples edges proportionally to μ_{ij} , applies Proposition 7.14 to the endpoints, and for each such update also samples a handful of random non-neighbors (≈ 5) to apply the repulsive gradient to — *negative sampling*, the standard trick for avoiding the $\Theta(n^2)$ full repulsion sum. The learning rate anneals linearly to zero over `n_epochs` (the local run used 500).

Initialization is not random: the points start at the *spectral embedding* of the graph — coordinates from the bottom nontrivial eigenvectors of the symmetric normalized graph Laplacian $L = I - D^{-1/2}MD^{-1/2}$ (where $M = (\mu_{ij})$ and D is its degree matrix). This is exactly the Laplacian-eigenmaps solution, a convex relaxation of the same “neighbors stay close” demand, and it starts SGD in a basin that already respects the coarse cluster structure.

7.5 What the Map Does and Does Not Mean

Proposition 7.15 (Isometry invariance). *Let $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ be any Euclidean isometry (rotation, reflection, translation, or a composition). Then $\mathcal{C}(Ty_1, \dots, Ty_n) = \mathcal{C}(y_1, \dots, y_n)$.*

Proof. $\mathcal{C}$ depends on the layout only through the pairwise distances $\|y_i - y_j\|$ (via ν_{ij}), and isometries preserve pairwise distances by definition. $\square$

Corollary 7.16 (The axes are meaningless). *The set of minimizers of $\mathcal{C}$ is closed under all Euclidean isometries. Which representative the algorithm returns is decided by the spectral initialization and the randomness of SGD — not by the data. In particular, no direction on the galaxy map (up, down, left, right) admits any interpretation, and separate runs on the same library may return maps differing by rotation, reflection, or more.*

What survives, and why one may trust it: μ_{ij} is built *only* from each point’s k nearest neighbors, and the objective’s attraction acts only on those pairs. Local neighborhoods are the signal; everything else — inter-cluster distances, cluster areas (Remark 7.9), global shape — is a side effect of repulsion and should be read with suspicion.

In the code

`compute_galaxy.py` runs `umap.UMAP(n_components=2, n_neighbors=15, min_dist=0.05, metric="cosine")` for positions. Post-processing: positions are centered and divided by the 99.5th percentile of coordinate magnitude — a robust rescale to roughly $[-1, 1]^2$ that one outlier moment cannot hijack. The client (`app.js`, `Galaxy`) and the drift view (`vibescenes.js`) treat the coordinates as opaque: everything they draw — hover neighborhoods, the song path, the flight camera — uses only relative position, as Corollary 7.16 demands.

7.6 Color as a Second Opinion

The star colors are not styling; they are a second, independent instance of the same mathematics. The pipeline runs UMAP *again* on the same 64-dimensional data with `n_components=3` and a looser `min_dist` of 0.3, yielding $z_i \in \mathbb{R}^3$, then maps robustly to RGB:

$$\text{rgb}_i = 60 + 195 \cdot \text{clip}\left(\frac{z_i - p_2}{p_{98} - p_2}, 0, 1\right),$$

where p_2, p_{98} are per-channel percentiles (the affine map lands in $[60, 255]^3$ so no star is too dark against the background).

Because the 3-D run shares the input graph-building mathematics but not the 2-D run's optimization trajectory, *hue similarity is a quasi-independent witness of timbral similarity*. When two nearby stars also share a color, two different embeddings agreed; when a star's color clashes with its neighborhood, the 2-D layout likely forced a compromise (Corollary 7.16 again: the plane is small).

Exercises

Exercise 7.1. Prove the identity $\|u - v\|^2 = 2d_{\cos}(u, v)$ for unit vectors, and exhibit three unit vectors in $\mathbb{R}^2$ violating the triangle inequality for $d_{\cos}$.

Exercise 7.2. Prove the subspace half of Theorem 7.3: among all k -dimensional subspaces S , total projected variance is maximized by the span of the top k eigenvectors. (Induct using the variational characterization on the orthogonal complement.)

Exercise 7.3. In Lemma 7.8, suppose point i has t neighbors tied exactly at distance ρ_i with $t > \log_2 k$. What goes wrong, and what does the condition mean geometrically? (The implementation clamps σ_i to a minimum value in this case.)

Exercise 7.4. Verify Proposition 7.14. Then derive the repulsive gradient $\partial[-(1 - \mu_{ij})\log(1 - \nu_{ij})]/\partial y_i$ and show it diverges as $r \rightarrow 0$ when $b < 1$. The implementation adds a small ε to r^2 ; explain what failure this prevents during SGD.

Exercise 7.5. Prove that $\mu_{ij} = w_{i \rightarrow j} + w_{j \rightarrow i} - w_{i \rightarrow j}w_{j \rightarrow i}$ satisfies $\max(w_{i \rightarrow j}, w_{j \rightarrow i}) \leq \mu_{ij} \leq \min(1, w_{i \rightarrow j} + w_{j \rightarrow i})$, and characterize when each bound is attained.

Exercise 7.6 (computational). Run `compute_galaxy.py` twice with different random seeds and overlay the two maps. Find the best-fitting rotation+reflection between them by solving the orthogonal Procrustes problem $\min_{Q \in \mathbb{R}^{d \times d}} \|Y_1 Q - Y_2\|_F$ (solution: $Q = UV^\top$ from the SVD of $Y_1^\top Y_2$). How much residual disagreement survives after aligning? Which regions of the map are stable, and are they the dense ones?

Exercise 7.7 (computational). Set `n_neighbors=5` and rebuild the galaxy. Describe what happens to (i) song threads, (ii) the number of visual clusters, (iii) bridges between genres. Explain each observation via Remark 7.9 and the role of k in Definition 7.6.

Chapter 8

Similarity Search and the Moment Index (*outline*)

“Find me moments that sound like this one” — across every song in the library, in milliseconds, along a chosen facet (whole mix, just the drums, just the vocals).

In the code

Status: outline. To be written against `moment_index.py`, `moments.py`, `descriptors.py`, `interactions.py`, and the `/api/similar` endpoint.

Planned contents:

- **Exact top- k retrieval.** Cosine scores as one matrix–vector product; complexity and memory layout; when brute force beats an index (spoiler: at this library’s scale, always).
- **Faceted similarity.** Per-stem embedding blocks (`emb_drums`, `emb_bass`, ...) as projections of the product space; facet weighting as a diagonal metric.
- **Interpretable descriptors.** The 18-dimensional descriptor vector (RMS, onset density, spectral centroid, active fraction per stem); z-scoring; mixing learned and hand-crafted metrics.
- **Diversity constraints.** The per-song result cap as a greedy approximation to maximal marginal relevance.
- **Approximate methods** (for the future large library): inverted-file indices, HNSW graphs, product quantization — what each trades away.

Part IV

Rendering

Chapter 9

The Mathematics of Rendering (*outline*)

Everything the visualizer draws is mathematics executed sixty times a second: procedural noise, divergence-free flow fields, radial symmetry groups, perspective projection, and color spaces.

In the code

Status: outline. To be written against `scene.js` (mandala shader, stage projection) and `vibescenes.js` (liquid advection, weather FBM, drift camera).

Planned contents:

- **Hash, value noise, and fBm.** Lattice noise with smoothstep interpolation (C^1 but not C^2 — and where that artifact is visible); fractional Brownian motion $\sum_k a^k n(2^k p)$, its spectrum, and the self-similarity parameter.
- **Curl noise.** In 2-D, the stream-function construction $v = (\partial_y \psi, -\partial_x \psi)$; *theorem*: $\nabla \cdot v = 0$ (incompressible flow — why the liquid view folds instead of draining); finite-difference curl and its step-size tradeoff.
- **Semi-Lagrangian advection.** The backward-lookup scheme $D_{t+1}(p) = D_t(p - v \Delta t)$; unconditional stability; numerical diffusion as implicit blur, exploited deliberately as dye decay.
- **Kaleidoscopes as group actions.** The dihedral group D_{12} acting on the plane; folding θ into a fundamental domain via $|\theta - s \lfloor \theta/s \rfloor|$; why chroma's 12 pitch classes make this the *honest* symmetry group for harmony.
- **Perspective projection.** The stage view's camera (`project()` in `scene.js`): yaw/pitch rotations as matrix products, similar triangles for the focal divide, painter's-algorithm depth sorting.
- **Color.** HSL/HSV as cylindrical re-parameterizations of RGB; hue arithmetic on the circle; why the vibe palette interpolates hue rather than RGB (avoiding desaturated midpoints); gamma and additive blending in the drift view.

- **Feedback systems.** Ping-pong framebuffers as discrete dynamical systems $D_{t+1} = f(D_t) + \text{input}_t$; contraction (decay < 1) as the guarantee the liquid never blows up.

Appendix A

Equal Temperament and Chroma

The small amount of music theory the codebase depends on, stated as mathematics.

A.1 Pitch is Logarithmic

The ear judges two pitch *intervals* equal when the frequency *ratios* are equal: the step from 110 to 220 Hz sounds like the step from 220 to 440 Hz. Perceived pitch is therefore a logarithm of frequency, and Western tuning discretizes that logarithm.

Definition A.1 (12-tone equal temperament). Fix the reference $A4 = 440$ Hz. The *semitone* is the ratio $2^{1/12}$, and the pitch number (MIDI number) of frequency $f > 0$ is

$$m(f) = 69 + 12 \log_2 \frac{f}{440},$$

so that $m(440) = 69$ and each octave (doubling) spans exactly 12 units. The nearest note is $\lfloor m(f) \rfloor$, and the error $m(f) - \lfloor m(f) \rfloor$, in units of 1/100 of a semitone, is measured in *cents*.

Remark A.2. $2^{1/12}$ is irrational, so no interval except the octave is acoustically pure: the equal-tempered fifth is $2^{7/12} \approx 1.4983$, not $3/2$. Equal temperament trades purity for symmetry — every key sounds the same — and it is exactly that symmetry the definitions below exploit.

Definition A.3 (Pitch class and chroma). The *pitch class* of frequency f is $c(f) = \lfloor m(f) \rfloor \bmod 12 \in \{0, \dots, 11\}$ ($0 = C$, $1 = C\sharp$, $\dots$, $11 = B$). A *chroma vector* of a spectrum $|X[k]|$ is the 12-vector that folds all octaves onto pitch classes:

$$\text{ch}[c] = \sum_{k: c(f_k)=c} |X[k]|, \quad c = 0, \dots, 11,$$

summing over bins k with frequencies f_k in a chosen band.

Chroma deliberately forgets octave and timbre, keeping only *harmony* — which notes are sounding. Two voicings of the same chord, octaves apart on different instruments, give nearly the same chroma vector.

In the code

Definition A.1 appears verbatim wherever the code names a note: `scene.js` computes `midi = Math.round(69 + 12 * Math.log2(hz / 440))` for the vocal and bass note labels, with `NOTE_NAMES[((midi % 12) + 12) % 12]` implementing $c(f)$ (the double-mod guards JavaScript's signed %). Definition A.3 is `computeChroma()`, folding analyser bins between 55 and 2000 Hz; the twelve values drive the mandala's twelve petals, making that visualization literally a plot of the chroma vector in polar coordinates. The per-bin nearest-note rounding is exact for high bins but aliases for low ones (Chapter 1, Exercise 1.6) — one reason chroma starts at 55 Hz rather than lower.

Exercises

Exercise A.1. Show that the map $f \mapsto c(f)$ satisfies $c(2f) = c(f)$, and that scaling f by the equal-tempered fifth $2^{7/12}$ advances the pitch class by 7 (mod 12). Since $\gcd(7, 12) = 1$, conclude that stacking fifths visits all 12 pitch classes — the circle of fifths as a statement about the group $\mathbb{Z}/12\mathbb{Z}$.

Exercise A.2. The bass string $E1$ has fundamental 41.2 Hz. Its second and third harmonics are 82.4 and 123.6 Hz. To which pitch classes do the first five harmonics round, and how many cents does each miss by? (This is why chroma computed on a bass-heavy band is polluted by harmonics — and why `computeChroma` prefers harmonic stems.)